探针验证报告：无 editor_sdk / 无 Office 生成可编辑 DOCX

> 日期：2026-10-11
> 目的：验证「便携 + 轻量」路线的**核心技术假设**是否成立
> 位置：spike/portable_export_probe/
> 结论：☑ **成立** —— 用 Python 标准库即可产出含原生 OMML 公式的 DOCX，Word 与 WPS 双端均可编辑

一、要验证的假设

「便携 + 轻量」路线成立的前提是：**能脱离 `editor_sdk` 与 Office 生成可编辑公式的 DOCX**。
若这一条不成立，整个便携路线就不成立。故先做探针，**不在生产代码上试错。**

编号	假设
H1	仅用 Python 标准库（`zipfile` + 手写 OOXML）即可产出合法 DOCX
H2	公式是**原生 OMML**（`word/document.xml` 内含 `<m:oMath>`）
H3	不含 `word/embeddings/*`（非 OLE 对象）、无图片冒充
H4	Word 能打开且不报错
H5	公式被识别为**原生 Word 公式**（`doc.OMaths.Count > 0`），而非图片/OLE
H6	**实际修改公式内部字符 → 保存 → 关闭 → 重开 → 改动保留**

> H6 的判定标准照搬项目铁律：**判定"公式可编辑"必须实际改公式内部字符→保存→重开→比对，**
> 不能用"有 OMML 节点"或"能打印"代替。

二、验证结果

步骤 1 • 生成与结构自检（`probe_generate.py`）

```
[OK] 已生成：probe_out.docx (1,282 字节)
[OK] 依赖：仅标准库 zipfile（未使用 python-docx / editor_sdk / Office）
```

--- 结 构 自 检 ---
parts: ['[Content_Types].xml', '_rels/.rels', 'word/document.xml']
H1 合 法 DOCX (3 个 必 需 part) : PASS
H2 原 生 OMML (<m:oMath> 数 量) : 4 PASS
H3 无 OLE 嵌 入 对 象 (word/embeddings/*) : 无 PASS
H3 无 图 片 冒 充 : 无 PASS
```

**一个 1,282 字节的 DOCX，含 4 条原生公式，零第三方依赖。**

### 步骤 2 • 双端可编辑验证（`probe_word_edit.py`）

| 引擎 | H4 打开 | H5 原生公式 | H6 改字符后保留 | 结论 |
|---|---|---|---|---|
| **Word** (Word.Application) | ☑ | ☑OMaths=4、InlineShapes=0 | ☑2→3，重开后仍在，公式数 4→4 | **PASS** |
| **WPS** (KWPS.Application) | ☑ | ☑OMaths=4、InlineShapes=0 | ☑2→3，重开后仍在，公式数 4→4 | **PASS** |

关 键 输 出 ( 两 端 一 致 ) :
```
[H5] OMaths.Count = 4 ← 原 生 Word 公 式 数 (>0 即 原 生)
[H5] InlineShapes = 0 ← 图 片 / 内 联 对 象 数 (应 为 0)
[H6] 公 式 1 修 改 前 : ' = \r \r2\r'
[H6] 就 地 修 改 : 第 7 个 字 符 '2' → '3'
[H6] 重 开 后 : ' = \r \r3\r' (OMaths=4)
[H6] 判 定 : PASS ← 字 符 改 动 保 留 (4 → 4 公 式 数 不 变)
```

### 步骤 3 • 渲染目视确认

导出的 PDF 渲染结果（`probe_render.png`）：

| 公式 | 内容 | 渲染情况 |
|---|---|---|
| 公式 1（上标） | $E = mc^2$ | ☑上标正确 |
| 公式 2（分式） | $\rho = m/V$ | ☑**有真正的分数线** |
| 公式 3（下标） | $v = v_0 + at$ | ☑下标正确 |
| 公式 4（含中文） | $浮 = (\rho/液) \cdot g \cdot V_{排}$ | ☑分式与中文均正常 |
```

说明：公式不是"看起来像公式的文本"，而是被 Word **按数学排版渲染**（分式有横线、上下标位置正确）。

三、一个值得记录的方法论教训

第一次 H6 测试失败了，但失败的是测法，不是产品。

- **错误测法**：用 `doc.OMaths(1).Range.Text = "新文本"` **整体覆写**公式。

  → 结果：公式被**拆散成 6 个**、内容丢失。

- **原因**：`Range.Text` 赋值是 COM 的**粗暴覆写**，会破坏公式内部结构。

  它测的是"用代码覆写公式"，不是"用户在公式里改一个字"。

- **正确测法**：`rng.Characters(7).Text = "3"` —— **就地替换单个字符**，这才等价于用户行为。

  → 结果：改动保留、公式结构完好。

> **教训**：验证"可编辑性"时，**验证动作必须等价于真实用户动作**。
> 用程序化的粗暴操作去测，得到的是"程序能不能覆写"，而不是"用户能不能改"。
> 这与项目既有教训同源——**"结构上覆盖"不算验证通过**。

四、结论

☑ 已证明（本次实测）

1. **便携路线的核心技术假设成立**：不依赖 `editor_sdk`、不依赖 Office，**用 Python 标准库**即可生成含原生 OMML 公式的 DOCX。
2. **公式是真原生**：Word 与 WPS 双端均识别为 `OMath`（`InlineShapes=0`，非图片、非 OLE）。
3. **公式真的可编辑**：双端均通过"改字符→保存→重开→比对"。
4. **体积极小**：最小样例 1,282 字节；生成逻辑**零第三方依赖**。

⚠ 尚未证明（需后续验证）

项	说明
完整版式保真	探针只验证了公式。项目历史踩过的坑（`<a:graphicData>` 缺 `uri`、TAB 控制字符、HTML 实体、制表位、题号双重渲染）**尚未在自研生成器上验证**

项	说明
图片行内位置	图片插入与行内定位未测（已知现状是"统一居中附题干后"）
LaTeX → OMML 的自动转换	探针是**手写 OMML**；真实场景需要从题库里的 LaTeX 自动转换（本项目公式为 LaTeX 存储）
跨机器便携	需在第二台机器验证（本次未做）
打包体积	未实测（预估 ~80–120 MB）
性能	未测（大批量题目的生成耗时）

下一步

1. **实现最小 LaTeX → OMML 转换器**，覆盖项目实际用到的符号集（可从 `bank` 的公式表统计频次）。
2. **扩展探针**：加图片、制表位、多题排版，逐个复现历史坑并解决。
3. **打包实测**：验证体积与跨机器便携。

附：探针文件

文件	说明
`spike/portable_export_probe/probe_generate.py`	生成 DOCX（仅标准库）+ 结构自检 H1–H3
`spike/portable_export_probe/probe_word_edit.py`	Word/WPS COM 验证 H4–H6（支持 `--engine word|wps`）
`spike/portable_export_probe/probe_out.docx`	探针产物（1,282 字节）
`spike/portable_export_probe/probe_render.png`	渲染目视确认